Campus Helpdesk - Project Overview and Security Design

Muaz Ahmed (if24b241) · Mohamed Amin Kasmi (if24b120) · Mahmoud Hashkeil (if25b198) · Mateo Lamthi (if23b263)

Live application: https://campushelpdeskteam.netlify.app/

What we built

Campus Helpdesk is a support ticketing application for a university campus. Students open private tickets about IT, facilities or academic problems, choose a category and priority, and follow the conversation until it is resolved. The point of the project was not only to build something usable, but to see how many of the weaknesses we covered in the course we could close off in a realistic application. Most of the security work therefore sits on the server, where it cannot be bypassed by editing the page in a browser.

The application has three roles. Students see their own tickets. Support staff see their own tickets and the tickets assigned to them, and can reply and change the status of assigned tickets. Administrators can assign tickets, manage roles, block accounts and sign other users out remotely. The admin area includes an overview of workload and unassigned tickets, and a view of recent security activity. Support staff can also leave internal notes on a ticket. Those notes never reach the requester, even when the requester is themselves a staff member, and adding one does not change the ticket update time that the requester can see.

Each ticket can carry up to five attachments. Students and staff can upload screenshots and documents, and anyone who is allowed to read the ticket can download them. Because file upload is one of the most common sources of vulnerabilities in student projects, we treated it as a security feature rather than a convenience, and describe the controls below.

Architecture and trust boundaries

The frontend is plain HTML, CSS and JavaScript with no framework and no third-party scripts. Netlify serves the static files and runs the Node.js API as a serverless function. PostgreSQL, hosted on Neon, stores users, tickets, replies, attachments, sessions, password-reset tokens, rate-limit counters and audit events.

The browser only ever talks to the API over HTTPS. It never receives database credentials and never connects to PostgreSQL directly. Every request is treated as untrusted and validated on the server before any business rule runs. Because functions are stateless and instances come and go, all shared state - sessions, rate counters, audit records - lives in the database. Versioned SQL migrations define the schema and are applied with a separate migration connection, so the runtime connection can be restricted to ordinary read and write privileges instead of schema changes.

Accounts and sessions

Passwords must be between 15 and 128 characters and are stored with scrypt using a random salt per password (N=32768, r=8, p=3). Verification uses a constant-time comparison. Failed logins return one generic message, and registration always returns the same response, so neither form can be used to discover which addresses have accounts.

A successful login creates a random 256-bit session token, but only its SHA-256 hash is written to the database. Sessions last eight hours, and signing in again replaces the previous session. Production cookies use the `__Host-` prefix with `Secure`, `HttpOnly`, `SameSite=Strict`, `Path=/` and no `Domain` attribute. Logging out, blocking an account or changing a role deletes the server-side session, so removing a cookie in the browser is not what ends access.

Protection against common weaknesses

SQL injection (CWE-89). Every query passes user data through PostgreSQL placeholders rather than building SQL strings. This covers login, ticket search, admin filters and the audit-log queries, and it holds for the attachment tables as well. Database constraints additionally limit roles, statuses, priorities and field lengths, so even a malformed insert is rejected by the schema.

Cross-site scripting (CWE-79). Tickets, replies, names and attachment filenames are stored as plain text and rendered with `textContent`, never `innerHTML`. An HTML payload submitted as a ticket simply appears as text. A restrictive Content Security Policy allows scripts and styles only from the application's own origin, and blocks inline scripts, objects and framing. The API responses carry the same policy, and uploads are always downloaded rather than displayed inline, which removes the usual image/SVG trick of getting markup executed.

Cross-site request forgery (CWE-352). Any request that changes state must come from the configured origin, must include an application-specific request header, and must use JSON. Authenticated writes also require a random CSRF token bound to the session, checked on the server. `SameSite=Strict` cookies add another layer. Login, registration and password recovery apply the origin and format checks even though no session exists yet.

Access control (CWE-862, CWE-639). Ticket reads, replies, attachment listing and downloads all check ownership or staff assignment on the server. Changing a ticket ID in the URL returns 404 rather than someone else's data, and each attempt is written to the audit log. Every admin route re-checks the administrator role before running a query. Registration accepts only the expected fields, so adding a `role` field to the request body is rejected as an unexpected field - we confirmed this against the deployed application, which returned HTTP 400. Internal notes are filtered inside the SQL query before the data ever reaches the browser.

Allowlists and blocking. Rather than trying to detect malicious-looking input, the API declares what is allowed: HTTP methods, request fields, categories, priorities, statuses,

roles, UUID formats and text lengths. Anything else is rejected. Account blocking is the explicit deny mechanism and revokes sessions immediately. We deliberately avoided blacklisting SQL keywords and HTML strings, because blocklists are easy to bypass and tend to reject legitimate support text.

File uploads. Only png, jpg, pdf, txt and csv files are accepted. The server checks the extension, then reads the actual magic bytes of the file and requires them to match, so an executable renamed to `.png` is refused. Text files must be valid UTF-8 without null bytes. Each file is limited to 3 MB, each ticket to five files, and each account to ten uploads per hour. The database enforces its own size limit as a second barrier, so the application limit cannot be bypassed by an unexpected code path. Filenames are stored only for display and never used to build a storage path; the download response forces `Content-Disposition: attachment` with `nosniff`, which prevents the browser from rendering a stored file as a page.

Password changes and recovery

Changing a password requires the current password and is rate-limited before the expensive scrypt computation runs, so a stolen session cannot be used to guess the password quickly. A successful change revokes every session and issues a fresh session and CSRF token. Password recovery uses a random 256-bit token that is stored only as a hash, expires after 30 minutes, works once and is superseded by a newer request. Completing a reset revokes all sessions and any outstanding reset links. The reset token travels in the URL fragment, which browsers do not send in HTTP requests, and the frontend removes it from the address bar as soon as the page loads. Actual email delivery depends on an operator-configured HTTPS webhook; without one, the application refuses to pretend the message was sent.

Availability and abuse resistance

Rate limiting is backed by atomic counters in PostgreSQL, so it works across serverless instances. Every request class has its own budget: 120 requests per source per minute overall, a separate per-account limit for reads, 20 authentication attempts per source per 15 minutes, 10 failed logins per email per 15 minutes, 20 ticket creations per hour, 60 replies per hour, 30 ticket updates per minute and 10 attachment uploads per hour. Only failed logins count against the per-email limit, so a valid sign-in never pushes a legitimate user closer to a lockout.

Other resource controls include a 16 KiB request-body limit (raised only on the attachment upload route, where the encoded file needs the room), bounded text fields, pagination with a capped offset, a small connection pool and five-second connection and query timeouts. Expired counters, sessions and reset tokens are cleaned up as traffic arrives. These measures address resource exhaustion (CWE-400, CWE-770), but we are careful not to claim DDoS protection: a distributed attack has to be handled at the hosting edge, with capacity monitoring and an operational response.

Logging and alerting

Audit events record login outcomes, denied access, ticket changes, permission updates, password operations, attachment uploads and administrative sign-outs. Each record includes the event type, actor, target, timestamp and request ID, and the request ID is returned to the client so a report can be matched to a log line. Changes that matter are written in the same transaction as their audit record, so if the record cannot be written the change is rolled back. Passwords, session tokens and ticket bodies are never written to the audit table.

The fifth failed login from a source within a 15-minute window raises a security alert. An optional HTTPS webhook delivers a redacted notification with a short timeout and redirects disabled, and records whether delivery is pending, delivered, failed or not configured. A failed delivery stays visible instead of being reported as success. Delivery is currently best effort; durable retries and independent monitoring are future work.

Verification

The application is checked by a GitHub Actions workflow on every push and pull request: the JavaScript syntax check, the unit tests, the API integration suite and a production-dependency audit. The unit tests cover password hashing, input validation, ticket permissions and cookie construction. The integration suite runs the real API against an embedded PostgreSQL engine in isolation and covers SQL and HTML payloads, cross-account isolation, CSRF and origin rejection, role and mass-assignment rejection, internal-note privacy, session rotation and revocation, the login cooldown, password change and reset flows, rate limits, body limits, and the full attachment pipeline - allowed types, magic-byte mismatch, ownership denial and the download round-trip. At the time of writing, four unit tests and nineteen integration checks pass, and `npm audit` reports no production vulnerabilities.

We also tested the deployed build directly. Unauthenticated calls to `/api/auth/me` return 401, a registration request from a foreign origin returns 403, a registration body containing `role: "admin"` returns 400, admin endpoints without a session return 401, and a forged 64-character session cookie returns 401. These results match what the code review predicts, though they are point checks rather than a full penetration test - that is the purpose of the partner exercise.

Repository: https://github.com/Hashkeil/CampusHelpdesk · **Implementation:** `server/api.mjs`, `server/security.mjs`, `server/db.mjs`, `public/app.js`, `netlify.toml`, `db/*.sql`, `tests/`